SIH 2026 • PROBLEM STATEMENT PS 26076 | MINISTRY OF EARTH SCIENCES (IMD)

Project Overview & Jury Viva Q&A Guide

Project Title: Personalized, Hyperlocal & Bilingual Homepage for 'Mausam' Weather Application

Target Users: Indian Citizens, Farmers, Asthmatic Patients, Commuters, Coastal Fishermen, Travelers

1. Executive Summary: What Are We Doing & Why?

The "Why" (The Real Problem)

Traditional weather apps suffer from **Context Blindness**. They present raw meteorological parameters—temperature, barometric pressure, dew point—which mean little to everyday citizens. An asthmatic individual does not know if 180 µg/m³ PM2.5 requires an N95 mask; a cotton farmer does not know if 72% relative humidity makes pesticide spraying wasteful; an evening runner does not know if heat index and ozone levels pose cardiovascular stress.

Furthermore, in India:

- **The Language Barrier:** Over 60% of rural and semi-urban citizens cannot comfortably parse technical English forecasts.
- **The Coastal Gap:** 7,516 km of coastline houses millions of fishermen and beachgoers who receive no sea-state or tide data unless they dig into specialized oceanographic websites.
- **The Data Opacity Gap:** Air quality reports rarely mention *which* sensor measured the pollution or its distance, leading to false security.

The "What" (Our Solution)

Under **SIH 2026 (PS 26076)** for the Ministry of Earth Sciences (IMD), we designed and built a **Personalized, Bilingual, Cross-Platform Homepage** for the official 'Mausam' application. It translates raw telemetry into **8 citizen-specific actionable personas**:

Citizen Persona	Key Data Signals Used	Actionable Output Delivered
1. Health-Conscious	CPCB National AQI, PM2.5, PM10, Pollen counts, UV	Mask recommendation, outdoor restriction alerts, nearest CAAQMS station name & distance attribution.
2. Outdoor Fitness	Temperature, Humidity, Heat Index, Air Quality	Comfort Score (0–100), optimal morning/evening running windows, heat stroke hydration warning.
3. Beachgoers & Surfers	INCOIS marine telemetry, tide tables, wave height	High/low tide timing, water safety flags. *Auto-triggered only when location is within 25 km of coast!*
4. Travelers	Multi-city watchlist, weather differentials, rain probability	Dynamic luggage packing checklist (umbrella, woolens, sunscreen), travel delays alert.

Citizen Persona	Key Data Signals Used	Actionable Output Delivered
5. Parents & Families	Heat index, UV index, rain forecast, school hours	Safe park play hours, child sunscreen advisories, school commute rain alerts.
6. Farmers & Agriculture	IMD Agromet district advisories, wind speed, rain probability	Pesticide spraying viability window, irrigation requirement advisory, crop damage risk flags.
7. Daily Commuters	Hourly precipitation trajectory, fog visibility, wind gusts	Safe departure times, waterlogging probability on roadways, safe helmet/drive precautions.
8. Event Planners	3-to-7 day precipitation probability, afternoon wind gusts	Outdoor viability risk score, indoor venue fallback recommendations.

2. System Architecture & Technical Mechanics: How Things Work

Three-Tier Architecture

1. **Client Layer (Flutter Cross-Platform):** Single codebase compiled to Web, Android APK, and iOS. Employs Material 3 design, responsive constraint managers, and an in-app View Mode Switcher.
2. **Backend Layer (Node.js Express BFF):** Acts as a Backend-For-Frontend proxy. Aggregates upstream government endpoints (IMD AWS, CPCB CAAQMS, INCOIS), caches geocoding responses in memory for 24 hours, and computes proximity.
3. **Zero-Drift Offline Twin (Dart Engine):** Contains an offline geocoding dictionary, identical GeoJSON coastline data, and local Equirectangular Haversine math. If the device loses internet connection, the app continues operating with full persona logic seamlessly.

The 25 km Coastal Proximity Algorithm

To respect MoES requirements, beach and marine cards must strictly appear only for locations within **25 kilometers** of the Indian shoreline. We implemented the **Equirectangular Projected Haversine Point-to-Segment Distance** algorithm across all 7,516 km of India's shoreline coordinates:

```
Δx = (lon₂ - lon₁) × cos((lat₁ + lat₂) / 2)
Δy = (lat₂ - lat₁)
Distance = Earth_Radius (6371 km) × √(Δx² + Δy²)
```

Zero-Drift Guarantee: Both backend (`utils/coastal.js`) and frontend (`coastal_detector.dart`) share the identical GeoJSON dataset (`india_coastline.json`) and identical mathematical projection. Verified benchmark test cases:

- **Mumbai:** 2.0 km to coast ≤ 25 km → COASTAL (TRUE)
- **Kochi:** 0.0 km to coast ≤ 25 km → COASTAL (TRUE)
- **Kottayam:** 22.0 km to coast ≤ 25 km → COASTAL (TRUE)
- **Varanasi:** 634.9 km to coast > 25 km → INLAND (FALSE)
- **New Delhi:** 865.0 km to coast > 25 km → INLAND (FALSE)

Bilingual Standardization

We avoided raw automated machine translation. All Air Quality index categories strictly adhere to the **Central Pollution Control Board (CPCB) official Hindi terminology**:

- Good (0–50) → **अच्छा**
- Satisfactory (51–100) → **संतोषजनक**
- Moderate (101–200) → **मध्यम**
- Poor (201–300) → **खराब**
- Very Poor (301–400) → **बहुत खराब**

- Severe (401–500) → **गंभीर**

Weather condition strings are similarly localized into vernacular Indian phrases (e.g. *Hazy Sunshine → धुंधली धूप, Thunderstorms → गरज के साथ बारिश*).

India-First Scope with Graceful Global Fallback

IMD's mandate is national, but citizens travel internationally. If a user searches for global cities (e.g. **London, Dubai, New York**):

- The location resolves and saves cleanly in `recentLocations` and `savedDestinations`.
- Core weather, temperature, humidity, and travel packing suggestions are fully computed.
- Regional Indian cards (Beach/Surfer and Agromet) display an informative notice: "*Available for Indian regions under Ministry of Earth Sciences mandate.*"

3. Comprehensive Jury Viva Q&A (20 Questions & Winning Answers)

Q1 Why do we need another weather app when Google Weather, AccuWeather, and IMD already exist?

Answer: Existing commercial apps are built on global models (GFS, ECMWF) that lack the high-resolution Doppler radar, Agromet advisory network, and CPCB CAAQMS sensor density that the Government of India operates. Furthermore, commercial apps treat all users identically. A farmer and a marathon runner see the exact same home screen. Our application uses official MoES/IMD data and translates it into **hyperlocal, role-based actionable advisories** with zero-advertising, complete data privacy, and vernacular language reach.

Q2 How does the application handle users in rural areas with poor internet connectivity?

Answer: We engineered a **Zero-Drift Offline Twin**. The Flutter client embeds a pre-indexed dictionary of Indian cities, districts, and coordinates, alongside the coastline GeoJSON. If a farmer opens the app in a village with no cellular reception, the app serves cached meteorological bulletins, computes coastal proximity locally in Dart, and displays offline status badges without crashing or showing blank error screens.

Q3 What is "Zero-Drift" in your coastline detection, and why is it important?

Answer: "Drift" occurs when two different implementations (e.g., JavaScript on backend and Dart on frontend) calculate slightly different results for the same coordinates due to floating-point rounding or differing Haversine algorithms. This could lead to an embarrassing situation where a city is marked "Coastal" online, but becomes "Inland" offline if Wi-Fi drops. We solved this by using the exact same GeoJSON coordinate dataset and mathematically identical Equirectangular Haversine projections on both sides, ensuring zero discrepancy.

Q4 How does the app determine whether someone is a "Beachgoer" or needs marine data?

Answer: Marine and beach telemetry is strictly location-gated by our 25-kilometer coastal threshold engine. If a user selects Mumbai or Kochi, the distance to the Arabian Sea is calculated as ≤ 25 km, which automatically activates the Beach & Surfer card with INCOIS tide tables and wave heights. If a user in New Delhi or Varanasi selects the Beach persona, the app displays a distance chip (e.g., "*Inland • 865 km to coast*") and recommends nearby coastal destinations instead of displaying irrelevant ocean tides.

Q5 What is "AQI Station Transparency" and how does your app implement it?

Answer: Most apps display a single city-wide AQI number (e.g., "Delhi AQI: 320"). In reality, air quality varies wildly across a city. Under PS 26076, we introduced Sensor Attribution: our Health Card explicitly renders the exact monitoring station name and its distance from the user (e.g., "*CAAQMS Central Station • 3.2 km away*"). This provides citizens with verifiable transparency into how close the physical monitoring station is to their neighborhood.

Q6 Can a user choose more than one persona at the same time?

Answer: Yes! Citizens have multifaceted lives—a school teacher can also be a daily commuter, a weekend runner, and a parent. In Settings, users can select any combination of the 8 personas. The homepage feed dynamically renders all active persona cards in prioritized order, and our sticky filter bar allows the user to filter down to a single persona with one tap.

Q7 What happens if a user searches for an international city like London or New York?

Answer: We follow the **India-First with Graceful Global Fallback** rule. Commercial geocoding or Nominatim resolves international coordinates. The app computes global weather, temperatures, and luggage packing recommendations (Traveler persona). However, regional cards governed by Indian government ministries (such as Agromet crop advisories and INCOIS beach safety) gracefully render an MoES mandate notice explaining that localized agricultural bulletins apply strictly to Indian territories.

Q8 How did you implement bilingual support? Is it just automated Google Translate?

Answer: No. Automated translators often produce awkward, inaccurate translations for meteorological terms. We built native Flutter `.arb` localization files (`app_en.arb` and `app_hi.arb`) with an official dictionary. Crucially, all 6 National Air Quality Index (NAQI) categories are mapped to the **official Central Pollution Control Board Hindi terminology**: *अच्छा, संतोषजनक, मध्यम, खराब, बहुत खराब,* and *गंभीर.*

Q9 How does your app run on both Mobile phones and Web browsers without altering code?

Answer: We engineered a **Dual-Mode Responsive Architecture** in Flutter:

- On mobile screens (< 720px), the UI renders 100% edge-to-edge with touch gestures and a dedicated bottom navigation bar.
- On desktop/web screens (≥ 900px), persona cards adapt into a 2-column grid to eliminate stretched cards, and the dashboard provides high-density vitals.
- We also built an **In-App View Mode Switcher**: clicking `[ 📱 Mobile ]` on a laptop screen renders an interactive smartphone chassis (420px) with rounded bezel, notch, and shadow, allowing effortless presentations on projectors.

Q10 What was the most critical bug you encountered during development, and how did you resolve it?

Answer: While implementing the bottom navigation bar for web, our entire weather body disappeared, leaving only a blank white screen with the navigation bar floating in the exact vertical center. By inspecting Flutter's `RenderScaffold` constraints, we diagnosed that wrapping `bottomNavigationBar` inside an unconstrained `Center` widget caused it to expand vertically to fill the entire screen height, squeezing the body's height to zero. We resolved it by constraining the navigation bar to a fixed `SizedBox(height: 80)` and aligning the body to `Alignment.topCenter` .

Q11 How do you calculate the "Comfort Index" for the Fitness and Parents personas?

Answer: Our algorithm combines Steadman's Apparent Temperature formula with humidity penalties and CPCB AQI deductions:
`Comfort_Score = 100 - (0.55 × |Temp - 22°C| × 2.5) - (Humidity > 60% ? (Humidity - 60) × 0.4 : 0) - (AQI > 100 ? (AQI - 100) × 0.15 : 0)` .
The result is normalized strictly between 0 and 100, providing an instant score on whether outdoor conditions are pleasant, acceptable, or hazardous.

Q12 How do you protect user privacy? Do you track user GPS coordinates on a remote server?

Answer: No GPS coordinates or user persona profiles are ever stored on our backend servers. User preferences, selected personas, recent searches, and language settings are stored exclusively on the user's client device using encrypted local storage (`SharedPreferences`).

Q13 What automated testing did you implement to ensure stability?

Answer: We have a 25-test automated unit suite that passes 100% in under 3 seconds:

- `coastal_test.dart` : Verifies sub-25km detection for coastal cities and rejection for inland cities.
- `comfort_index_test.dart` : Verifies comfort formulas, high-AQI running restrictions, and packing list triggers.
- `location_model_test.dart` : Tests global scope gating and recents caching for international cities (London).
- `weather_translations_test.dart` : Verifies all 6 CPCB Hindi NAQI terms and weather condition dictionaries.
- `weather_model_test.dart` : Tests JSON serialization/deserialization and persona mapping integrity.

Q14 How can this platform scale to all 22 official scheduled languages of India?

Answer: Our localization architecture uses standard Flutter `l10n` ARB templates. Adding a new language (e.g. Marathi, Tamil, Bengali, Telugu) requires creating a single key-value file (e.g. `app_mr.arb`). The UI automatically generates typed localization bindings with zero refactoring of widget code.

Q15 How does the app prevent API rate limiting when thousands of citizens search simultaneously?

Answer: Three levels of protection: 1. **Search Debouncing:** The search modal buffers keystrokes for 350ms, eliminating redundant queries. 2. **BFF In-Memory Cache:** The Node.js proxy caches geocoding responses in memory for 24 hours. 3. **Client-Side Fallback:** Top Indian cities are resolved instantly from the client's local dictionary without issuing network requests.

Q16 How does the Farmer persona help reduce agricultural input costs?

Answer: Chemical pesticides washed away by sudden rain represent an annual loss of hundreds of crores for Indian farmers. Our Agromet card analyzes 6-hour precipitation probability and wind speed thresholds (< 15 km/h) to give farmers an explicit "*Safe for Chemical Spraying*" or "*Do Not Spray - Rain Expected*" advisory, directly saving agricultural costs.

Q17 What happens if the GPS permission is denied by the user?

Answer: The app never blocks user access. If GPS is denied or unavailable, the app falls back to New Delhi (default) or the user's last chosen city, and prompts the user to use the instant search modal to type any village, district, or city name.

Q18 What is your deployment strategy for the Ministry of Earth Sciences?

Answer: The app is production-ready. The web frontend compiles to a static bundle (`build/web`) deployable on government NIC/MeitY cloud servers or CDNs with zero hosting cost. The mobile client compiles directly to an Android APK ready for publication on the Google Play Store and Mobile Seva AppStore.

Q19 What font optimization was done to ensure fast loading on 2G/3G networks?

Answer: In our production build, Flutter's font tree-shaking reduced the MaterialIcons font payload by **98.7%** (from 1.64 MB down to just 21.4 KB), ensuring sub-second bundle load times even on slow 2G and 3G cellular connections in rural India.

Q20 What is the single biggest takeaway you want the jury to remember about this project?

Answer: "*We did not simply build a weather dashboard; we built an intelligent decision-support system for 1.4 billion Indian citizens. By combining official MoES/IMD data with 8 citizen personas, deep Hindi localization, sub-25km coastal precision, and offline zero-drift reliability, we have made meteorological science actionable for every Indian.*"